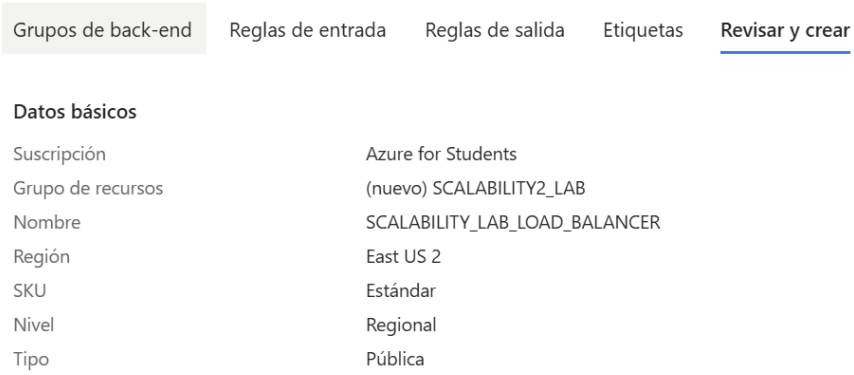
Parte 2 - Escalabilidad horizontal

Crear el Balanceador de Carga

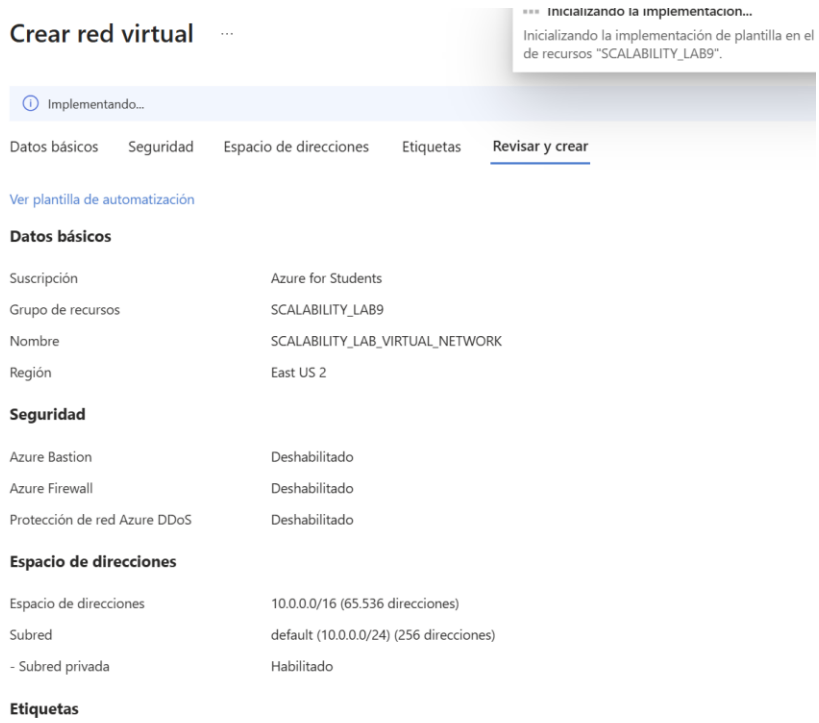
Antes de continuar puede eliminar el grupo de recursos anterior para evitar gastos adicionales y realizar la actividad en un grupo de recursos totalmente limpio.

1. El Balanceador de Carga es un recurso fundamental para habilitar la escalabilidad horizontal de nuestro sistema, por eso en este paso cree un balanceador de carga dentro de Azure tal cual como se muestra en la imágen adjunta.

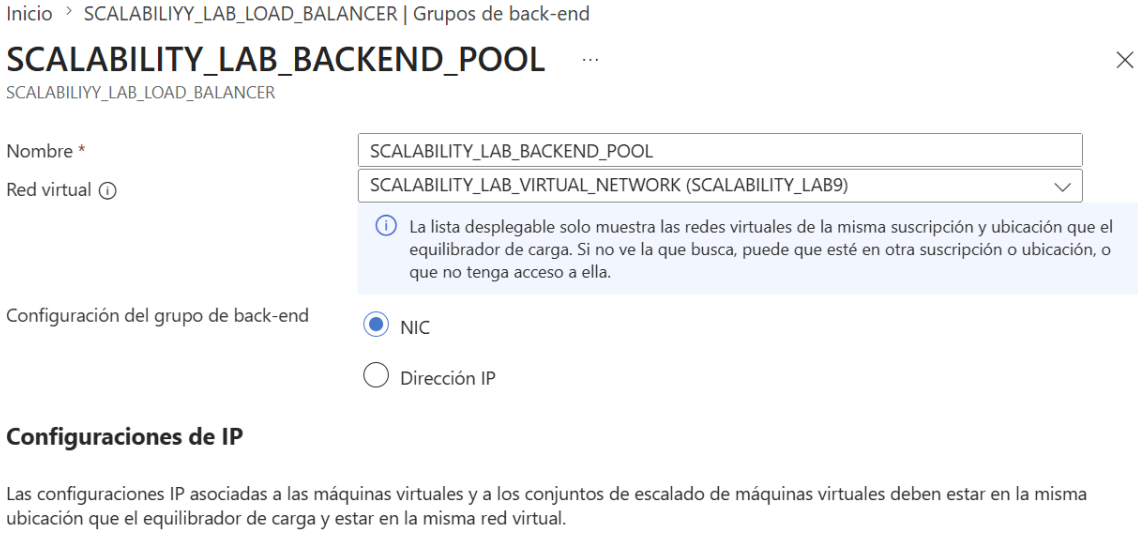
→



- Creamos la virtual network



- Creamos un backend-pool



- Creamos un health probe

Agregar sondeo de estado

SCALABILIYY_LAB_LOAD_BALANCER

Los sondeos de estado se usan para comprobar el estado de una instancia de grupo de back-end. Si el sondeo de estado no puede obtener respuesta de una instancia de back-end, no se enviarán nuevas conexiones a esa instancia de back-end hasta que el sondeo de estado vuelva a realizarse correctamente.

Nombre *	SCALABILITY_LAB_HEALTH_PROBE
Protocolo *	TCP
Puerto *	3000
Intervalo (segundos) *	5
Usado por *	No se usa

- Creamos un load balancing rule

Inicio > SCALABILIYY_LAB_LOAD_BALANCER | Reglas de equilibrio de carga

Agregar regla de equilibrio de carga

SCALABILIYY_LAB_LOAD_BALANCER

Nombre *	SCALABILITY_LAB_LOA_BALANCING_RULE
Versión de IP *	(•) IPv4 ( ) IPv6
Dirección IP de front-end *	frontend-ip-fibonacci (20.75.111.10)
Grupo de back-end *	SCALABILITY_LAB_BACKEND_POOL
Protocolo	(•) TCP ( ) UDP
Puerto *	80
Puerto de back-end *	3000
Sondeo de estado *	SCALABILITY_LAB_HEALTH_PROBE (TCP:3000) Crear nuevo
Persistencia de la sesión	Ninguna — La persistencia de sesión especifica que el tráfico de un cliente debe ser controlado por la misma máquina virtual en el grupo de back-end durante una sesión. Más información.
Tiempo de espera de inactividad (minutos) *	4
Habilitar el restablecimiento de TCP	[x]

Crear las maquinas virtuales (Nodos)

Ahora vamos a crear 3 VMs (VM1, VM2 y VM3) con direcciones IP públicas standar en 3 diferentes zonas de disponibilidad. Después las agregaremos al balanceador de carga.

1. En la configuración básica de la VM guíese por la siguiente imágen. Es importante que se fije en la "Avaiability Zone", donde la VM1 será 1, la VM2 será 2 y la VM3 será 3.
2. En la configuración de networking, verifique que se ha seleccionado la *Virtual Network* y la *Subnet* creadas anteriormente. Adicionalmente asigne una IP pública y no olvide habilitar la redundancia de zona.

3. Para el Network Security Group seleccione "avanzado" y realice la siguiente configuración. No olvide crear un *Inbound Rule*, en el cual habilite el tráfico por el puerto 3000. Cuando cree la VM2 y la VM3, no necesita volver a crear el *Network Security Group*, sino que puede seleccionar el anteriormente creado.

4. Ahora asignaremos esta VM a nuestro balanceador de carga, para ello siga la configuración de la siguiente imágen.

→

Redes

Red virtual	SCALABLITY_LAB_VIRTUAL_NETWORK
Subred	default (10.1.0.0/24)
Redes aceleradas	Desactivado
Eliminar NIC al eliminar la VM	Deshabilitado

organizar y administrar todos los recursos.

Suscripción * Azure for Students

Grupo de recursos * SCALABILITY_LAB09

Crear nuevo

Detalles de instancia

Azure ha detectado que hay otras recomendaciones de región, tamaño y zona que pueden dar lugar a mayores posibilidades de implementaciones correctas. Para implementar sin problemas, adopte cualquiera de las recomendaciones sugeridas.
Vea la recomendación.

Nombres de máquina virtual VM1-1, VM2-2, VM3-3

3 máquinas virtuales se crearán con los nombres que se muestran arriba.
Editar nombres

Región * (South America) Chile Central

Implementación en una zona extendida de Azure

Opciones de disponibilidad Zona de disponibilidad

Opciones de zona
- Zona autoseleccionada
  Elija hasta 3 zonas de disponibilidad, una máquina virtual por zona
- Zona seleccionada por Azure (versión preliminar)
  Permitir que Azure asigne la mejor zona para sus necesidades

Zona de disponibilidad * Zonas 1, 2, 3

En función de la selección de zona, colocaremos 3 máquinas virtuales, una en

Crear una máquina virtual

Ayuda para crear una máquina virtual de b

Validación superada

Recursos asociados

Máquinas virtuales	(nuevo) VM1-1, VM2-2, VM3-3
IP públicas	Ninguno
Azure Load Balancer	(nuevo) SCALABILITY_LAB_LOAD_BALANCER

Datos básicos

Suscripción	Azure for Students
Grupo de recursos	SCALABILITY_LAB09
Región	Chile Central
Opciones de disponibilidad	Zona de disponibilidad
Opciones de zona	Zona autoseleccionada
Zona de disponibilidad	1, 2, 3
Tipo de seguridad	Estándar
Imagen	Ubuntu Server 24.04 LTS - Gen2
Arquitectura de VM	x64
Tamaño	Standard B1ls (1 vcpu, 0.5 GiB de memoria)
Habilitar hibernación	No
Tipo de autenticación	Clave pública SSH
Nombre de usuario	vm1
Formato de clave SSH	RSA
Nombre de par de claves	VM1-1_key
Puertos de entrada públicos	SSH
Azure de acceso puntual	No

Discos

Tamaño del disco del SO	Valor predeterminado de la imagen

5. Finalmente debemos instalar la aplicación de Fibonacci en la VM. para ello puede ejecutar el conjunto de los siguientes comandos, cambiando el nombre de la VM por el correcto

→

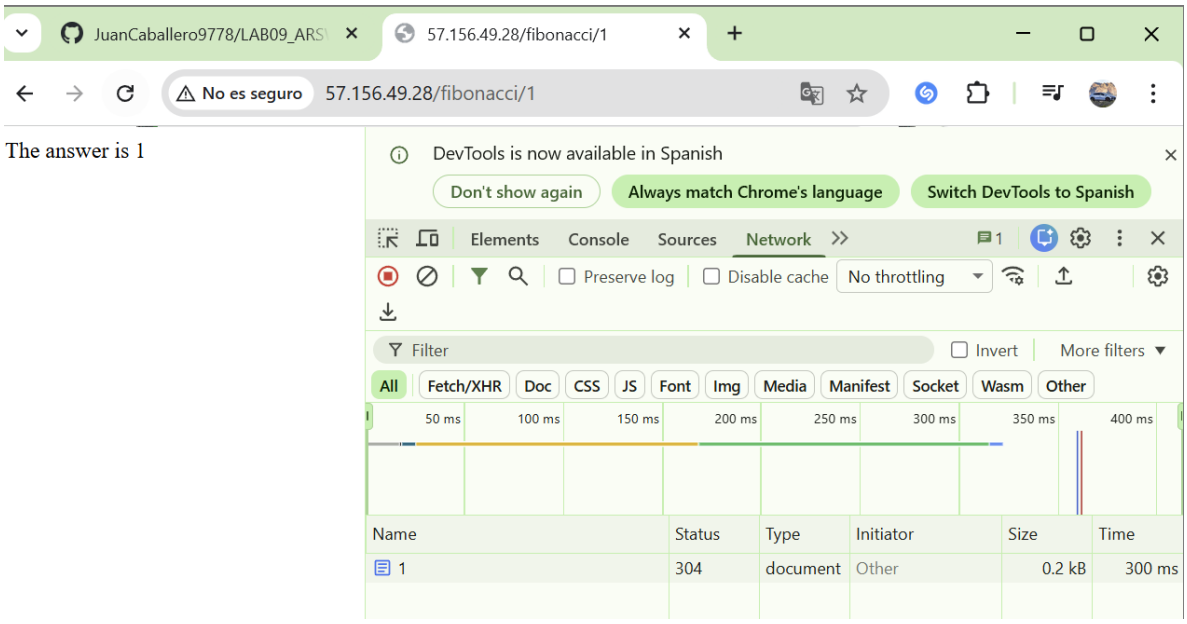
```
vm1@VM1-1:~/LAB09_ARSW/FibonacciApp$ forever start FibonacciApp.js
warn:    --minUptime not set. Defaulting to: 1000ms
warn:    --spinSleepTime not set. Your script will exit if it does not stay up for at l
east 1000ms
info:    Forever processing file: FibonacciApp.js
(node:10015) Warning: Accessing non-existent property 'padLevels' of module exports ins
ide circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:10015) Warning: Accessing non-existent property 'padLevels' of module exports ins
ide circular dependency
vm1@VM1-1:~/LAB09_ARSW/FibonacciApp$ forever list
(node:10051) Warning: Accessing non-existent property 'padLevels' of module exports ins
ide circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:10051) Warning: Accessing non-existent property 'padLevels' of module exports ins
ide circular dependency
info:    Forever processes running
data:        uid  command       script          forever pid   id logfile
      uptime
data:    [0] amr9 /usr/bin/node FibonacciApp.js 10026   10036    /home/vm1/.forever/amr
9.log 0:0:1:8.188000000000002
vm1@VM1-1:~/LAB09_ARSW/FibonacciApp$
```

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
^C^C
vm1@VM2-2:~/LAB09_ARSW/FibonacciApp$ node -v
v18.19.1
vm1@VM2-2:~/LAB09_ARSW/FibonacciApp$ npm -v
9.2.0
vm1@VM2-2:~/LAB09_ARSW/FibonacciApp$ npm install
npm WARN old lockfile
npm WARN old lockfile The package-lock.json file was created with an old version of npm,
npm WARN old lockfile so supplemental metadata must be fetched from the registry.
npm WARN old lockfile
npm WARN old lockfile This is a one-time fix-up, please be patient...
npm WARN old lockfile

added 51 packages, and audited 52 packages in 4s

7 vulnerabilities (3 low, 4 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
vm1@VM2-2:~/LAB09_ARSW/FibonacciApp$ sudo npm install -g forever
npm WARN deprecated source-map-url@0.4.1: See https://github.com/lydell/source-map-url#deprecated
npm WARN deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Pleas
e update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm WARN deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests
 by a key value, which is much more comprehensive and powerful.
npm WARN deprecated urix@0.1.0: Please see https://github.com/lydell/urix#deprecated
npm WARN deprecated resolve-url@0.2.1: https://github.com/lydell/resolve-url#deprecated
npm WARN deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm WARN deprecated source-map-resolve@0.5.3: See https://github.com/lydell/source-map-resolve#deprecated

added 289 packages in 1m

50 packages are looking for funding
  run `npm fund` for details
vm1@VM2-2:~/LAB09_ARSW/FibonacciApp$ forever start FibonacciApp.js
warn:    --minUptime not set. Defaulting to: 1000ms
warn:    --spinSleepTime not set. Your script will exit if it does not stay up for at least 1000ms
info:    Forever processing file: FibonacciApp.js
(node:9734) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:9734) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
vm1@VM2-2:~/LAB09_ARSW/FibonacciApp$ forever list
(node:9765) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:9765) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
info:    Forever processes running
data:        uid  command       script          forever pid  id logfile                    uptime
data:    [0] HwAH /usr/bin/node FibonacciApp.js 9745    9755    /home/vm1/.forever/HwAH.log 0:0:0:6.602
vm1@VM2-2:~/LAB09_ARSW/FibonacciApp$
```

Probar el resultado final de nuestra infraestructura

1. Porsupuesto el endpoint de acceso a nuestro sistema será la IP pública del balanceador de carga, primero verifiquemos que los servicios básicos están funcionando, consuma los siguientes recursos:

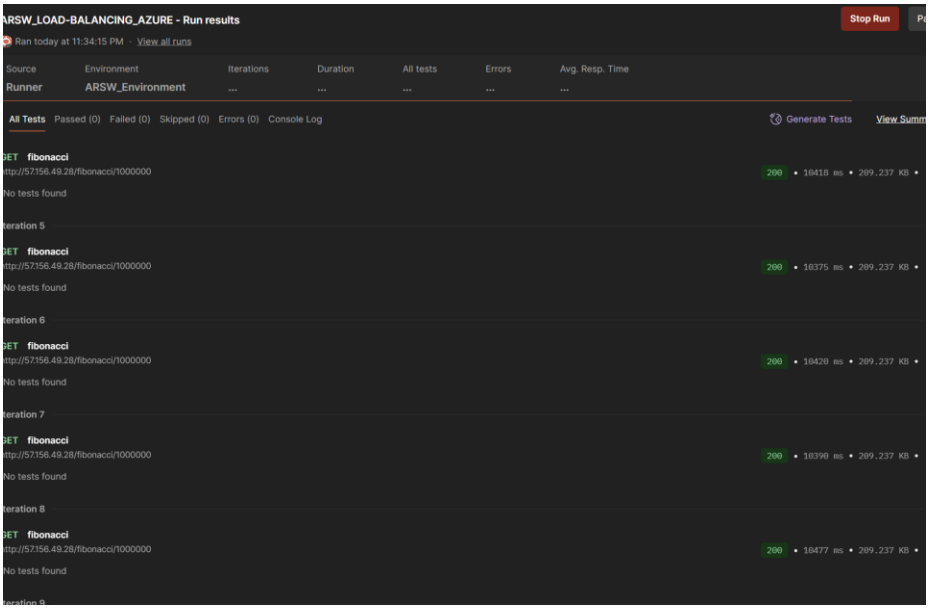
→



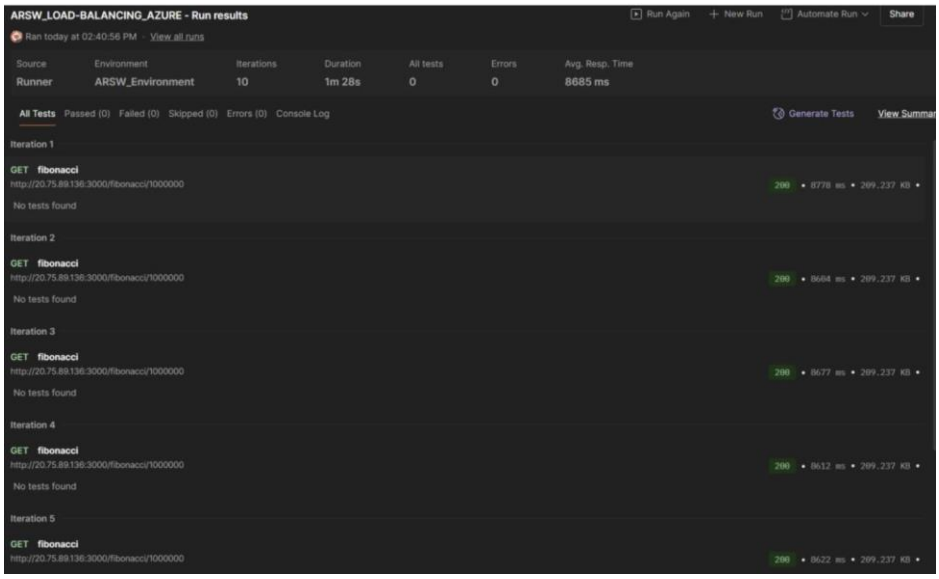
2. Realice las pruebas de carga con newman que se realizaron en la parte 1 y haga un informe comparativo donde contraste: tiempos de respuesta, cantidad de peticiones respondidas con éxito, costos de las 2 infraestrucruras, es decir, la que desarrollamos con balanceo de carga horizontal y la que se hizo con una maquina virtual escalada.

→

Balanceado



No balanceado



Preguntas

- ¿Cuáles son los tipos de balanceadores de carga en Azure y en qué se diferencian?, ¿Qué es SKU, qué tipos hay y en qué se diferencian?, ¿Por qué el balanceador de carga necesita una IP pública?

→ Azure ofrece varias soluciones de balanceo, las cuales se dividen principalmente por la capa del modelo OSI en la que operan:

- **Azure Load Balancer:**
    - **Capa:** Capa 4 (Transporte - TCP/UDP).
    - **Función:** Distribuye el tráfico entrante basado en reglas de puerto e IP. Es ideal para tráfico de alta velocidad y baja latencia. Es el que utilizaste para tu laboratorio.
- **Azure Application Gateway:**
    - **Capa:** Capa 7 (Aplicación - HTTP/HTTPS).
    - **Función:** Permite enrutamiento basado en URLs, terminación SSL y afinidad de sesión (cookies). Es "inteligente" porque entiende el contenido de la petición web.
- **Azure Front Door:**
    - **Capa:** Capa 7 (Global).
    - **Función:** Es un balanceador global que acelera la entrega de contenido usando la red global de Microsoft. Ideal para aplicaciones web distribuidas en múltiples regiones.

SKU significa *Stock Keeping Unit*. En Azure, define el "nivel" o "categoría" de servicio, lo que determina sus características, límites de escala y costo.

- Tipos de SKU para Load Balancer:
    - SKU Básico (Basic): Limitado, diseñado para aplicaciones pequeñas, no tiene soporte para redundancia de zona ni monitoreo avanzado. Se usa principalmente en entornos de desarrollo o pruebas.
    - SKU Estándar (Standard): Es el estándar de producción. Soporta redundancia de zona, diagnósticos detallados, conjuntos de escalado de máquinas virtuales y permite una seguridad mucho más robusta. Este es el que se recomienda para infraestructuras profesionales.

Diferencia clave: El Estándar es más costoso pero ofrece alta disponibilidad (99.99%) y capacidad de escalado que el Básico no puede igualar.

Sin la IP pública, la aplicación sería una isla inaccesible desde Internet, y sin los SKUs adecuados (como el Estándar), no podríamos garantizar la robustez que requiere una arquitectura profesional de balanceo de carga horizontal.

- ¿Cuál es el propósito del *Backend Pool*?

→ El Backend Pool convierte un conjunto de máquinas individuales en un cluster capaz de trabajar coordinadamente, permitiéndote escalar de forma horizontal (añadiendo más VMs) o vertical sin que el usuario final perciba cambios en la arquitectura.

- ¿Cuál es el propósito del *Health Probe*?

→ Sin el *Health Probe*, el balanceador sería un sistema "ciego" que seguiría enviando usuarios a máquinas apagadas o aplicaciones congeladas. La sonda es el componente que hace que tu infraestructura sea tolerante a fallos.

- ¿Cuál es el propósito de la *Load Balancing Rule*? ¿Qué tipos de sesión persistente existen, por qué esto es importante y cómo puede afectar la escalabilidad del sistema?.

→ La Load Balancing Rule es el cerebro operativo que garantiza que el tráfico llegue al puerto correcto de forma segura. Su uso debe ser equilibrado; el exceso de persistencia puede impedir que el balanceador distribuya el tráfico de manera eficiente, lo que limita la verdadera capacidad de escalado horizontal al sobrecargar innecesariamente nodos específicos.

- ¿Qué es una *Virtual Network*? ¿Qué es una *Subnet*? ¿Para qué sirven los *address space* y *address range*?

→ La Virtual Network actúa como la infraestructura de red primaria, proporcionando aislamiento y seguridad. Las Subnets permiten la segmentación jerárquica de esta red, facilitando la organización y la aplicación de políticas de seguridad. El Address Space define el límite total de direccionamiento IP disponible para la red, mientras que los Address Ranges permiten delegar porciones de dicho espacio a subredes específicas,

asegurando una administración eficiente y evitando conflictos de IP en arquitecturas de red complejas.

- ¿Qué son las *Availability Zone* y por qué seleccionamos 3 diferentes zonas?. ¿Qué significa que una IP sea *zone-redundant*?

→ La selección de tres zonas de disponibilidad permite implementar una arquitectura multi-zonal que mitiga riesgos de fallos catastróficos a nivel de centro de datos físico. Al combinar esto con una IP zone-redundant, garantizamos que el punto de entrada de nuestra aplicación (el balanceador de carga) permanezca accesible desde Internet, independientemente de la salud de una zona específica.

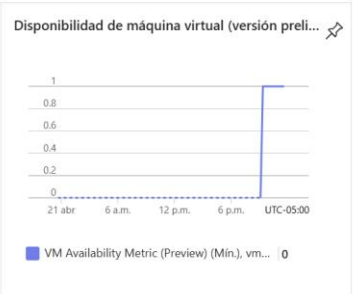
- ¿Cuál es el propósito del *Network Security Group*?

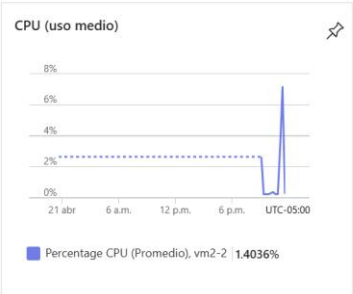
→ El Network Security Group es el componente de seguridad encargado de la segmentación y el control de acceso a nivel de red. Su propósito es definir un perímetro de seguridad estricto, permitiendo el flujo de datos únicamente a través de canales autorizados y puertos específicos.

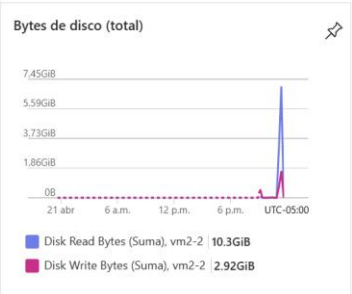
- Informe de newman 1 (Punto 2)

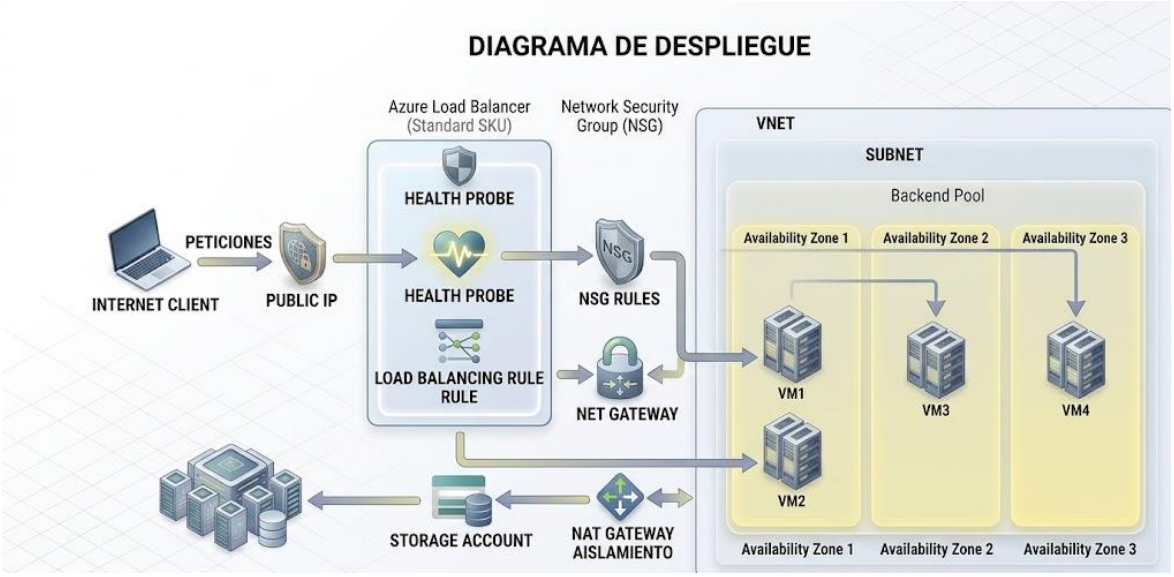
→ Conclusión: Aunque en pruebas de baja concurrencia (10 peticiones secuenciales) los tiempos de respuesta son similares, la infraestructura balanceada demuestra superioridad en términos de escalabilidad y tolerancia a fallos. La pequeña diferencia en el tiempo promedio se atribuye a la latencia de red inherente al balanceador, la cual es un costo necesario para garantizar la alta disponibilidad. La solución balanceada permite que el sistema crezca de forma horizontal, siendo capaz de atender múltiples usuarios concurrentes sin que el tiempo de respuesta se degrade de forma exponencial, tal como ocurriría en una arquitectura vertical ante picos de demanda.

Costo CPU muchísimo menor por que distribuye cargas entre las maquinas que estén disponibles







Así que el Balanceado: El tiempo de respuesta se mantendrá estable. Aunque cada petición individual tarde lo mismo, el sistema no colapsará.

- Presente el Diagrama de Despliegue de la solución.